LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK

“SISTEM PERBANKAN MINI:
DARI OBJECT, CLASS, DAN CONSTRUCTOR HINGGA POLYMORPHISM”

disusun oleh:

Dennis Shauqi Akbar

2511533020

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

BAB I – PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Ruang Lingkup Praktikum 1
1.4 Lingkungan Pengembangan 2
1.5 Sistematika Laporan 2
BAB II – PRAKTIKUM 1: OBJECT, CLASS, CONSTRUCTOR 3
2.1 Langkah Kerja Praktikum 3
2.2 Tugas/Latihan Praktikum 6
BAB III – PRAKTIKUM 2: RELASI ANTAR OBJEK (ASOSIASI) 11
3.1 Langkah Kerja Praktikum 11
3.2 Tugas/Latihan Praktikum 13
BAB IV – PRAKTIKUM 3: KEAMANAN & INTEGRITAS DATA (ENKAPSULASI) 17
4.1 Langkah Kerja Praktikum 17
4.2 Tugas/Latihan Praktikum 20
BAB V – PRAKTIKUM 4: EKSPANSI PRODUK KEUANGAN (INHERITANCE) 24
5.1 Langkah Kerja Praktikum 24
5.2 Tugas/Latihan Praktikum 27
BAB VI – PRAKTIKUM 5: TRANSAKSI DINAMIS (POLYMORPHISM) 31
6.1 Langkah Kerja Praktikum 31
6.2 Tugas/Latihan Praktikum 35
BAB VII – KESIMPULAN 40
7.1 Perkembangan Sistem 40
7.2 Kesimpulan 40
7.3 Repositori Kode 41

BAB I – PENDAHULUAN

1.1 Latar Belakang

Laporan akhir ini disusun sebagai rangkuman dari lima pertemuan praktikum mata kuliah Pemrograman Berorientasi Objek (PBO). Seluruh praktikum dikerjakan secara berkesinambungan pada satu studi kasus yang sama, yaitu Sistem Perbankan Mini berbasis konsol (CLI) yang dikembangkan menggunakan bahasa pemrograman Java pada IDE Eclipse. Pada setiap pertemuan, sistem tersebut diperluas dengan satu konsep PBO baru sehingga perkembangan kode dari pertemuan pertama hingga kelima dapat diikuti secara utuh.

1.2 Tujuan

Tujuan penyusunan laporan akhir ini adalah sebagai berikut.

- Merangkum langkah kerja, kode program, dan hasil pengujian dari praktikum pertama sampai kelima.
- Menunjukkan keterkaitan antarpraktikum dalam membangun satu sistem perbankan mini yang utuh.
- Menjelaskan penerapan konsep object, class, constructor, asosiasi, enkapsulasi, inheritance, dan polymorphism pada kasus nyata.

1.3 Ruang Lingkup Praktikum

Tabel 1.1 memuat ringkasan topik dan konsep utama pada setiap praktikum beserta nama package yang digunakan di dalam project PraktikumPBO.

No	Package	Topik Praktikum	Konsep Utama
1	pekan1	Object, Class, Constructor	Class, objek, constructor, ArrayList
2	pekan2	Relasi Antar Objek (Asosiasi)	Asosiasi one-to-many, class Transaksi
3	pekan3	Keamanan & Integritas Data (Enkapsulasi)	Modifier private, getter, PIN, otentikasi
4	pekan4	Ekspansi Produk Keuangan (Inheritance)	extends, super, upcasting, downcasting

No	Package	Topik Praktikum	Konsep Utama
5	pekan5	Transaksi Dinamis (Polymorphism)	Overriding, dynamic method dispatch, parameter polimorfik

Tabel 1.1 Ringkasan lima praktikum

1.4 Lingkungan Pengembangan

- Bahasa pemrograman: Java.
- IDE: Eclipse, dengan Java Project bernama PraktikumPBO.
- Antarmuka: konsol (Command Line Interface) berbasis menu.
- Pengelolaan kode: repository GitHub shauqiakbar/PrakPBO_2511533020.

1.5 Sistematika Laporan

Bab I membahas latar belakang, tujuan, dan ruang lingkup. Bab II sampai Bab VI secara berurutan memuat hasil Praktikum 1 sampai Praktikum 5, masing-masing berisi langkah kerja, tugas atau latihan, serta hasil pengujian. Bab VII berisi kesimpulan keseluruhan praktikum.

BAB II – PRAKTIKUM 1: OBJECT, CLASS, CONSTRUCTOR

Praktikum pertama memperkenalkan dasar pemrograman berorientasi objek, yaitu class sebagai cetak biru (blueprint), objek sebagai wujud nyata dari class, dan constructor sebagai method khusus yang dijalankan saat objek dibuat. Studi kasus yang digunakan adalah rekening bank sederhana.

2.1 Langkah Kerja Praktikum

a. Persiapan Project

Aplikasi Eclipse dibuka, lalu dibuat Java Project baru bernama PraktikumPBO. Di dalam project tersebut dibuat package pekan1 sebagai tempat penyimpanan seluruh source code pada pertemuan ini. Selanjutnya dibuat dua file program di dalam package pekan1, yaitu Rekening.java dan Main.java.

b. Class Rekening

Class Rekening berfungsi sebagai blueprint dari objek rekening bank. Class ini memiliki tiga atribut, yaitu nomorRekening dan namaPemilik bertipe String, serta saldo bertipe double. Constructor Rekening(String nomor, String nama, double saldoAwal) digunakan untuk menginisialisasi ketiga atribut tersebut sekaligus mencetak pesan konfirmasi saat objek pertama kali dibuat. Method setorTunai(double nominal) menambah saldo apabila nominal yang dimasukkan lebih besar dari 0; jika tidak, transaksi ditolak dengan pesan kesalahan.

Rekening.java | module-info.java | main.java | pertemuan3.java

```java
package pekan1;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public boolean cocokDenganNomor(String nomor) {
        return this.nomorRekening.equals(nomor);
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("----------------------");
    }
}
```

Gambar 2.1 Kode class Rekening

c. Class Main

Class Main merupakan program utama yang menjalankan sistem perbankan mini. Perulangan while (isRunning) digunakan agar menu terus ditampilkan selama program belum ditutup oleh pengguna. Variabel akunAktif bertipe Rekening menyimpan rekening yang sedang aktif dengan nilai awal null karena belum ada rekening yang dibuat. Pada case 1, pengguna membuat rekening baru yang langsung dijadikan akunAktif, sedangkan case 2 memproses setor tunai dengan pengecekan terlebih dahulu apakah akunAktif masih null.

```java
package pekan1;

import java.util.ArrayList;
import java.util.Scanner;

public class main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null;
        ArrayList<Rekening> rekeningAktif = new ArrayList<>();
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();

                    akunAktif = new Rekening(no, nama, saldo);
                    rekeningAktif.add(akunAktif);
                    System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor);
                    }
                    break;

                case 3:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka rekening!");
                    } else {
                        System.out.print("Masukkan nominal tarik: ");
                        double tarik = input.nextDouble();
                        akunAktif.tarikTunai(tarik);
                    }
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka rekening!");
```

Gambar 2.2 Kode class Main (bagian 1)

```java
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 5:
                    if (rekeningAktif.isEmpty()) {
                        System.out.println("Belum ada rekening yang terdaftar!");
                    } else {
                        System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
                        String cariNomor = input.nextLine();
                        boolean ditemukan = false;
                        for (Rekening r : rekeningAktif) {
                            if (r.cocokDenganNomor(cariNomor)) {
                                akunAktif = r;
                                ditemukan = true;
                                System.out.println("Berhasil! Akun aktif sekarang atas nama " + r.namaPemilik
                                        + " (No. Rekening: " + r.nomorRekening + ")");
                                break;
                            }
                        }
                        if (!ditemukan) {
                            System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
                        }
                    }
                    break;

                case 0:
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima kasih!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

Gambar 2.3 Kode class Main (bagian 2)

d. Hasil Pengujian

Pengujian awal dilakukan dengan memilih menu 1 untuk membuka rekening baru atas nama dennis dengan saldo awal Rp65.000. Program berhasil membuat objek rekening, menambahkannya ke dalam daftar, lalu kembali menampilkan menu utama.

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 8348320
Masukkan Nama Pemilik: dennis
Masukkan Saldo Awal: 65000
Rekening atas nama dennis berhasil dibuat dengan saldo Rp65000.0
Rekening berhasil ditambahkan. Total rekening: 1

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

Gambar 2.4 Output pembuatan rekening baru

2.2 Tugas/Latihan Praktikum

a. Implementasi Fitur Tarik Tunai

Tugas pertama adalah menambahkan method tarikTunai(double nominal) pada class Rekening dengan aturan bahwa saldo tidak boleh menjadi minus dan nominal penarikan minimal adalah Rp10.000. Apabila aturan dilanggar, program tidak boleh mengalami crash atau error, melainkan menampilkan peringatan ke konsol. Method ini kemudian diterapkan pada menu nomor 3 di class Main.

Method tarikTunai() melakukan dua pengecekan secara berurutan. Pertama, jika nominal kurang dari 10.000, transaksi langsung ditolak dengan pesan bahwa minimal penarikan adalah Rp10.000. Kedua, jika nominal lebih besar dari saldo, transaksi ditolak dengan pesan saldo tidak mencukupi beserta jumlah saldo saat ini. Apabila kedua syarat terpenuhi, saldo dikurangi dan dicetak pesan keberhasilan beserta saldo terbaru.

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}
```

Gambar 2.5 Kode method tarikTunai pada class Rekening

Pada class Main, case 3 terlebih dahulu memeriksa apakah akunAktif masih bernilai null. Jika belum ada rekening aktif, ditampilkan pesan error. Jika sudah

ada, pengguna diminta memasukkan nominal penarikan yang kemudian diproses melalui method tarikTunai() milik objek akunAktif.

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan nominal tarik: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Gambar 2.6 Kode case 3 (Tarik Tunai) pada class Main

Pengujian dilakukan sebanyak tiga kali dengan skenario berbeda. Percobaan pertama menggunakan nominal Rp5.000 yang kurang dari batas minimal.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 5000
Transaksi Gagal: Minimal nominal penarikan Rp10000
```

Gambar 2.7 Output penarikan di bawah batas minimal (Rp5.000)

Percobaan kedua menggunakan nominal Rp100.000 yang melebihi saldo yang tersedia, yaitu Rp85.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 100000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp85000.0
```

Gambar 2.8 Output penarikan melebihi saldo (Rp100.000)

Percobaan ketiga menggunakan nominal Rp10.000 yang memenuhi kedua syarat.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 10000
Tarik tunai Rp10000.0 berhasil. Saldo saat ini: Rp75000.0
```

Gambar 2.9 Output penarikan berhasil (Rp10.000)

Ketiga hasil pengujian tersebut menunjukkan bahwa method tarikTunai() berjalan sesuai aturan, yaitu menolak transaksi yang tidak memenuhi syarat tanpa membuat program mengalami crash.

b. Tantangan Multi-Akun

Pada versi awal, aplikasi hanya dapat menyimpan satu akun melalui variabel akunAktif sehingga pembukaan rekening kedua akan menimpa rekening pertama. Tantangan pada bagian ini adalah mengubah arsitektur penyimpanan menggunakan struktur data Collection (ArrayList), menambahkan objek rekening baru ke dalam ArrayList setiap kali menu "Buka Rekening" dipilih, serta menambahkan menu "Ganti Akun" sebagai bonus.

Pada class Rekening ditambahkan method cocokDenganNomor(String nomor) yang membandingkan nomorRekening milik objek dengan nomor masukan pengguna menggunakan equals(). Method ini mengembalikan true apabila kedua nomor sama persis dan false apabila berbeda, sehingga dapat dipakai untuk mencari rekening tertentu pada fitur Ganti Akun.

```
public boolean cocokDenganNomor(String nomor) {
    return this.nomorRekening.equals(nomor);
}
```

Gambar 2.10 Kode method cocokDenganNomor

Pada class Main, penyimpanan rekening diubah dari satu variabel objek menjadi ArrayList<Rekening> bernama rekeningAktif. Dengan demikian, setiap rekening baru pada case 1 tidak lagi menimpa rekening sebelumnya, melainkan ditambahkan menggunakan rekeningAktif.add(akunAktif).

```
ArrayList<Rekening> rekeningAktif = new ArrayList<>();
```

Gambar 2.11 Deklarasi ArrayList<Rekening>

Fitur Ganti Akun pada case 5 pertama-tama memeriksa apakah daftar rekening kosong. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program menelusuri daftar menggunakan perulangan for-each sambil mencocokkan nomor dengan r.cocokDenganNomor(cariNomor). Apabila ditemukan, objek tersebut dijadikan akunAktif yang baru, flag ditemukan diubah menjadi true, dan perulangan

dihentikan dengan break. Jika sampai akhir perulangan tidak ditemukan kecocokan, program menampilkan pesan bahwa nomor rekening tidak ditemukan.

```
case 5:
    if (rekeningAktif.isEmpty()) {
        System.out.println("Belum ada rekening yang terdaftar!");
    } else {
        System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
        String cariNomor = input.nextLine();
        boolean ditemukan = false;
        for (Rekening r : rekeningAktif) {
            if (r.cocokDenganNomor(cariNomor)) {
                akunAktif = r;
                ditemukan = true;
                System.out.println("Berhasil! Akun aktif sekarang atas nama " + r.namaPemilik
                        + " (No. Rekening: " + r.nomorRekening + ")");
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
        }
    }
    break;
```

Gambar 2.12 Kode case 5 (Ganti Akun)

Pengujian dimulai dengan membuka rekening kedua atas nama akmal. Total rekening bertambah menjadi 2 tanpa menimpa rekening pertama.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 7364839
Masukkan Nama Pemilik: akmal
Masukkan Saldo Awal: 100000
Rekening atas nama akmal berhasil dibuat dengan saldo Rp100000.0
Rekening berhasil ditambahkan. Total rekening: 2
```

Gambar 2.13 Output pembuatan rekening kedua

Selanjutnya dilakukan perpindahan akun menggunakan nomor rekening yang terdaftar.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening yang ingin diaktifkan: 7364839
Berhasil! Akun aktif sekarang atas nama akmal (No. Rekening: 7364839)
```

Gambar 2.14 Output ganti akun dengan nomor yang terdaftar

Terakhir dicoba berpindah akun dengan nomor rekening yang tidak terdaftar.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 5
Masukkan nomor rekening yang ingin diaktifkan: 39874050
Gagal: Nomor rekening 39874050 tidak ditemukan.

Menu Utama:
```

Gambar 2.15 Output ganti akun dengan nomor yang tidak terdaftar

Hasil pengujian menunjukkan bahwa program berhasil menyimpan lebih dari satu rekening tanpa saling menimpa serta dapat berpindah antarrekening berdasarkan nomor rekening yang dicari.

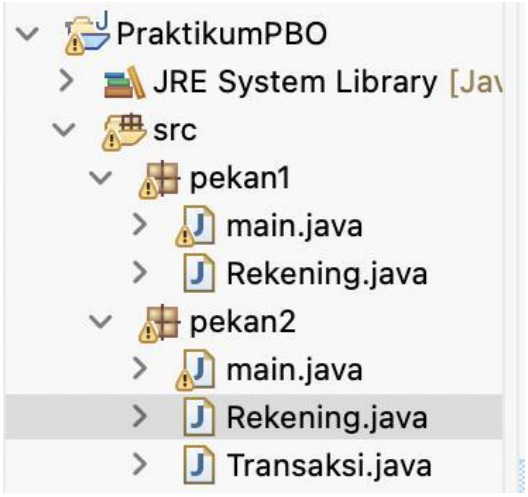
BAB III – PRAKTIKUM 2: RELASI ANTAR OBJEK (ASOSIASI)

Praktikum kedua mengembangkan sistem perbankan mini agar mampu menyimpan riwayat transaksi. Hal ini dicapai melalui penerapan relasi asosiasi antarobjek, yaitu satu objek Rekening dapat memiliki banyak objek Transaksi (one-to-many).

3.1 Langkah Kerja Praktikum

a. Persiapan Package

Aplikasi Eclipse dibuka kembali, kemudian pada project PraktikumPBO dibuat package baru bernama pekan2 sebagai tempat penyimpanan source code praktikum kedua. Package ini berisi tiga file, yaitu main.java, Rekening.java, dan Transaksi.java.



Gambar 3.1 Struktur project PraktikumPBO

b. Class Transaksi

Class Transaksi dibuat sebagai model dari satu catatan transaksi pada sebuah rekening. Class ini memiliki tiga atribut, yaitu idTransaksi dan jenis bertipe String, serta nominal bertipe double. Constructor Transaksi(String id, String jenis, double nominal) menginisialisasi ketiga atribut tersebut dengan memanfaatkan keyword this untuk membedakan atribut milik objek dari parameter. Atribut jenis memiliki dua kemungkinan nilai, yaitu Kredit untuk penyetoran dan Debit untuk penarikan. Method cetakDetail() menampilkan satu baris informasi lengkap berisi ID, jenis, dan nominal transaksi.

```java
package pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    //Contructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;

    }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }
}
```

Gambar 3.2 Kode class Transaksi

c. Relasi pada Class Rekening

Pada class Rekening ditambahkan atribut ArrayList<Transaksi> riwayatTransaksi yang merepresentasikan relasi has-a (asosiasi one-to-many). Atribut ini wajib diinisialisasi di dalam constructor menggunakan this.riwayatTransaksi = new ArrayList<>() agar koleksi siap digunakan saat objek rekening pertama kali dibuat. Tanpa inisialisasi tersebut, program akan mengalami NullPointerException ketika menambahkan data transaksi karena ArrayList masih bernilai null.

```java
package pekan2;
import java.util.*;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi Asosiasi (1-to-Many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib Menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

```

Gambar 3.3 Relasi asosiasi pada class Rekening

d. Perekaman Riwayat pada Method setorTunai

Method setorTunai(double nominal) diperbaiki agar tidak hanya menambah saldo, tetapi juga mencatat riwayatnya. Apabila nominal lebih besar dari 0, saldo ditambah, lalu dibentuk ID transaksi berprefiks "TRX-S-" yang digabungkan dengan System.currentTimeMillis() agar setiap ID unik. Selanjutnya dibuat objek Transaksi baru berjenis "Kredit" yang langsung ditambahkan ke riwayatTransaksi melalui add().

```
23    public void setorTunai(double nominal) {
24        if (nominal > 0) {
25            saldo += nominal;
26            // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
27            String idTrx = "TRX-S-" + System.currentTimeMillis();
28            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
29            riwayatTransaksi.add(trxBaru);
30
31            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
32        } else {
33            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
34        }
35    }
```

Gambar 3.4 Kode method setorTunai dengan perekaman riwayat

3.2 Tugas/Latihan Praktikum

a. Tugas 1: Integrasi Rekam Jejak Penarikan

Method tarikTunai(double nominal) dimodifikasi agar turut merekam objek Transaksi seperti pada setorTunai(). Perekaman diletakkan di dalam blok else, yaitu blok yang hanya dieksekusi apabila kedua syarat penarikan terpenuhi (nominal minimal Rp10.000 dan tidak melebihi saldo). Penempatan ini penting agar transaksi hanya tercatat ketika penarikan benar-benar berhasil sehingga penarikan yang gagal tidak masuk ke riwayat. ID transaksi berprefiks "TRX-T-" dengan jenis "Debit".

```
37    public void tarikTunai(double nominal) {
38        if (nominal < 10000) {
39
40            System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
41        } else if (nominal > saldo) {
42
43            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
44        } else {
45            saldo -= nominal;
46           // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
47            String idTrx = "TRX-T-" + System.currentTimeMillis();
48            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
49            riwayatTransaksi.add(trxBaru);
50
51            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
52        }
53    }
```

Gambar 3.5 Kode method tarikTunai dengan perekaman riwayat

b. Tugas 2: Fitur Cetak Mutasi Rekening

Method cetakMutasi() dibuat pada class Rekening untuk menampilkan seluruh riwayat transaksi. Method ini terlebih dahulu memeriksa apakah riwayatTransaksi kosong dengan isEmpty(). Jika kosong, ditampilkan pesan bahwa belum ada transaksi. Jika tidak, dilakukan perulangan for-each pada setiap objek Transaksi dan memanggil cetakDetail() miliknya. Pada bagian ini terlihat interaksi antarobjek: objek Rekening tidak mencetak data transaksi secara langsung, melainkan mendelegasikan tugas tersebut kepada objek Transaksi yang dimilikinya.

```
55    public void cetakMutasi() {
56        if (riwayatTransaksi.isEmpty()) {
57            System.out.println("Belum ada transaksi pada rekening ini");
58        } else {
59            for (Transaksi t : riwayatTransaksi) {
60                t.cetakDetail();
61
62            }
63
64        }
65    }
```

Gambar 3.6 Kode method cetakMutasi

Pada class Main ditambahkan menu baru "6. Cetak Mutasi (Riwayat)". Case 6 memeriksa apakah akunAktif bernilai null; jika tidak, program memanggil akunAktif.cetakMutasi().

```
93              case 6:
94              if (akunAktif == null) {
95                  System.out.println("Error: Anda belum membuka rekening!");
96              } else {
97                  akunAktif.cetakMutasi();
98
99          }
100             break;
```

Gambar 3.7 Kode case 6 (Cetak Mutasi) pada class Main

c. Hasil Pengujian

Pengujian pertama dilakukan pada rekening yang baru dibuat dan belum memiliki transaksi.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Belum ada transaksi pada rekening ini
```

Gambar 3.8 Output cetak mutasi pada rekening tanpa transaksi

Selanjutnya dilakukan dua kali penyetoran masing-masing sebesar Rp35.000 dan Rp15.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 35000
Setor tunai Rp35000.0 berhasil. Saldo saat ini: Rp85000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 15000
Setor tunai Rp15000.0 berhasil. Saldo saat ini: Rp100000.0
```

Gambar 3.9 Output dua kali setor tunai

Kemudian dilakukan satu kali penarikan sebesar Rp50.000 yang berhasil karena memenuhi batas minimal dan tidak melebihi saldo.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 50000
Tarik tunai Rp50000.0 berhasil. Saldo saat ini: Rp50000.0
```

Gambar 3.10 Output tarik tunai Rp50.000

Setelah ketiga transaksi tersebut, menu Cetak Mutasi dipilih kembali untuk melihat seluruh riwayat.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
ID: TRX-S-1789835859356 | Jenis: Kredit | Nominal: Rp35000.0
ID: TRX-S-1789835864914 | Jenis: Kredit | Nominal: Rp15000.0
ID: TRX-T-1789835916890 | Jenis: Debit | Nominal: Rp50000.0
```

Gambar 3.11 Output cetak mutasi setelah tiga transaksi

Hasil pengujian menunjukkan bahwa setiap transaksi setor dan tarik yang berhasil secara otomatis membentuk objek Transaksi baru dan menyimpannya ke riwayatTransaksi milik rekening bersangkutan. Dua transaksi setor tercatat sebagai

Kredit dengan prefiks ID TRX-S-, sedangkan transaksi tarik tercatat sebagai Debit dengan prefiks TRX-T-. Hal ini membuktikan bahwa relasi asosiasi one-to-many antara Rekening dan Transaksi telah berhasil diimplementasikan.

BAB IV – PRAKTIKUM 3: KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)

Praktikum ketiga membahas enkapsulasi, yaitu prinsip menyembunyikan data internal sebuah class dan hanya membuka akses melalui method yang terkontrol. Pada praktikum sebelumnya, atribut seperti saldo masih dapat diubah langsung dari luar class sehingga validasi pada setorTunai() dan tarikTunai() berpotensi dilewati. Praktikum ini menutup celah tersebut dan menambahkan lapisan keamanan berupa PIN. Seluruh kode pada pertemuan ini disimpan pada package pekan3.

4.1 Langkah Kerja Praktikum

a. Penguncian Class Transaksi

Seluruh atribut pada class Transaksi, yaitu idTransaksi, jenis, dan nominal, diubah menjadi private agar tidak dapat diakses atau diubah langsung dari luar class. Objek transaksi bersifat immutable, artinya tidak boleh berubah setelah dibuat. Oleh karena itu, class ini hanya dilengkapi getter, yaitu getIdTransaksi(), getJenis(), dan getNominal(), tanpa setter sama sekali. Dengan rancangan ini, sebuah transaksi yang sudah tercatat tidak dapat dimanipulasi oleh kode lain.

```
1  package pekan3;
2
3  public class Transaksi {
4      // 1. Mengubah semua atribut menjadi private
5      private String idTransaksi;
6      private String jenis;
7      private double nominal;
8
9      // Constructor
10     public Transaksi(String id, String jenis, double nominal) {
11         this.idTransaksi = id;
12         this.jenis = jenis;
13         this.nominal = nominal;
14     }
15
16     // 2. Hanya menyediakan Getter (Read-only)
17     public String getIdTransaksi() { return idTransaksi; }
18     public String getJenis() { return jenis; }
19     public double getNominal() { return nominal; }
20
21     public void cetakDetail() {
22         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
23     }
24 }
```

Gambar 4.1 Kode class Transaksi dengan atribut private

b. Penguncian Class Rekening dan Penambahan PIN

Perlakuan serupa diterapkan pada class Rekening. Seluruh atributnya, termasuk saldo, dikunci menjadi private agar tidak dapat dimanipulasi dari luar class, misalnya melalui perintah akunAktif.saldo = 9999999. Atribut baru bernama pin juga dibuat private karena termasuk data sensitif.



```java
package pekan3;
import java.util.*;

public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif

    private ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat");
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // 4. Method Otentikasi Internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal: Minimal nominal penarikan Rp10000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
```

Gambar 4.2 Kode class Rekening dengan atribut private dan PIN (bagian 1)



```java
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }
    }

    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi t : riwayatTransaksi) {
                t.cetakDetail();
            }
        }
    }

    public boolean cocokDenganNomor(String nomor) {
        return this.nomorRekening.equals(nomor);
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("---------------------");
    }
}
```

Gambar 4.3 Kode class Rekening dengan atribut private dan PIN (bagian 2)

Constructor dimodifikasi agar menerima parameter pinAwal, lalu memvalidasi panjangnya. Jika PIN terdiri atas tepat enam karakter, PIN tersebut dipakai. Jika tidak, program menampilkan peringatan dan otomatis menggunakan PIN default “123456”. Perlu dicatat bahwa validasi ini hanya memeriksa panjang PIN, bukan memeriksa apakah seluruh karakternya berupa angka. Untuk akses data, getter hanya disediakan bagi nomor rekening dan nama pemilik. Pemeriksaan PIN dilakukan melalui method otentikasi(String inputPin), yang menjadi satu-satunya

cara untuk memeriksa kecocokan PIN dengan membandingkannya menggunakan equals() dan mengembalikan nilai boolean.

c. Hasil Pengujian Pembuatan Rekening

Percobaan pertama membuka rekening atas nama dennis dengan nomor 2273245 dan saldo awal Rp150.000, menggunakan PIN enam digit yang valid, yaitu 212223. Rekening berhasil dibuat tanpa peringatan dan total rekening menjadi 1.

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 2273245
Masukkan Nama Pemilik: dennis
Masukkan Saldo Awal: 150000
Masukkan Pin (6 digit): 212223
Rekening atas nama dennis berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 1
```

Gambar 4.4 Output pembuatan rekening dengan PIN valid (212223)

Percobaan kedua membuka rekening atas nama akmal dengan nomor 9427302 dan saldo awal Rp300.000, menggunakan PIN 89314724 yang terdiri atas delapan digit sehingga tidak memenuhi aturan.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 9427302
Masukkan Nama Pemilik: akmal
Masukkan Saldo Awal: 300000
Masukkan Pin (6 digit): 89314724
Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456
Rekening atas nama akmal berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 2
```

Gambar 4.5 Output pembuatan rekening dengan PIN tidak valid (89314724)

Hasil kedua percobaan menunjukkan bahwa validasi panjang PIN pada constructor bekerja dengan benar. PIN enam digit diterima apa adanya, sedangkan

PIN yang tidak sesuai aturan diganti otomatis dengan PIN default 123456 disertai pesan peringatan, tanpa membuat program berhenti karena error. Rekening akmal tetap terbentuk dan total rekening menjadi 2.

d. Alasan setSaldo() dan getPin() Tidak Dibuat

Method setSaldo() sengaja tidak dibuat karena saldo tidak boleh diubah secara bebas dari luar class. Perubahan saldo hanya boleh terjadi melalui mekanisme yang sah, yaitu setorTunai() dan tarikTunai(), yang masing-masing memiliki validasi berupa nominal harus positif, batas minimal penarikan, dan kecukupan saldo. Seandainya setSaldo() disediakan, seluruh validasi tersebut dapat dilewati begitu saja.

Method getPin() juga sengaja tidak dibuat karena PIN merupakan data rahasia yang hanya boleh dicocokkan, bukan dibaca. Inilah alasan disediakannya method otentikasi() yang hanya mengembalikan hasil perbandingan (true atau false) tanpa pernah menampilkan nilai PIN itu sendiri.

4.2 Tugas/Latihan Praktikum

a. Tarik Tunai dengan Otentikasi PIN

Tugas pada bagian ini adalah mengamankan antarmuka konsol dengan otentikasi. Pada case 3 (Tarik Tunai), program terlebih dahulu memeriksa apakah akunAktif masih bernilai null untuk mencegah error. Jika sudah ada rekening aktif, pengguna diminta memasukkan PIN yang kemudian diverifikasi melalui akunAktif.otentikasi(pinYangDiinput). Apabila hasilnya true, pengguna baru diminta memasukkan nominal penarikan dan transaksi diproses. Apabila false, transaksi langsung dibatalkan dengan pesan “Akses Ditolak: PIN yang Anda masukkan salah!” tanpa memberi kesempatan memasukkan nominal.

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput)) {
            System.out.print("Masukkan nominal tarik: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Gambar 4.6 Kode case 3 (Tarik Tunai) dengan otentikasi PIN

Pengujian diawali dengan setor tunai sebesar Rp15.000 pada rekening yang sedang aktif, yaitu rekening akmal, sehingga saldonya naik dari Rp300.000 menjadi Rp315.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 15000
Setor tunai Rp15000.0 berhasil. Saldo saat ini: Rp315000.0
```

Gambar 4.7 Output setor tunai Rp15.000

Selanjutnya dicoba penarikan Rp80.000 dengan PIN 123456. PIN ini merupakan PIN default yang menggantikan PIN delapan digit pada rekening akmal. Penarikan berhasil dan saldo menjadi Rp235.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan PIN: 123456
Masukkan nominal tarik: 80000
Tarik tunai Rp80000.0 berhasil. Saldo saat ini: Rp235000.0
```

Gambar 4.8 Output tarik tunai dengan PIN benar (123456)

Percobaan terakhir menggunakan PIN yang salah, yaitu 372473. Program tidak sempat meminta nominal penarikan dan transaksi langsung dibatalkan. Hal ini membuktikan bahwa lapisan otentikasi berhasil mencegah transaksi yang tidak sah.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan PIN: 372473
Akses Ditolak: PIN yang Anda masukkan salah!
```

Gambar 4.9 Output tarik tunai dengan PIN salah (372473)

b. Cetak Mutasi dengan Otentikasi PIN

Logika yang sama diterapkan pada case 6 (Cetak Mutasi), yaitu meminta dan memverifikasi PIN sebelum riwayat transaksi ditampilkan. Dengan demikian, riwayat transaksi yang bersifat pribadi tidak dapat dilihat oleh orang yang tidak mengetahui PIN.

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiinput1 = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiinput1)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Gambar 4.10 Kode case 6 (Cetak Mutasi) dengan otentikasi PIN

Pada pengujian dengan PIN yang benar (123456), program menampilkan dua transaksi pada rekening akmal, yaitu Kredit sebesar Rp15.000 dari setor tunai dan Debit sebesar Rp80.000 dari tarik tunai.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Masukkan PIN: 123456
ID: TRX-S-1790408819562 | Jenis: Kredit | Nominal: Rp15000.0
ID: TRX-T-1790408843972 | Jenis: Debit | Nominal: Rp80000.0
```

Gambar 4.11 Output cetak mutasi dengan PIN benar (123456)

Sebaliknya, pada pengujian dengan PIN yang salah (235472), akses ke data mutasi ditolak sepenuhnya.

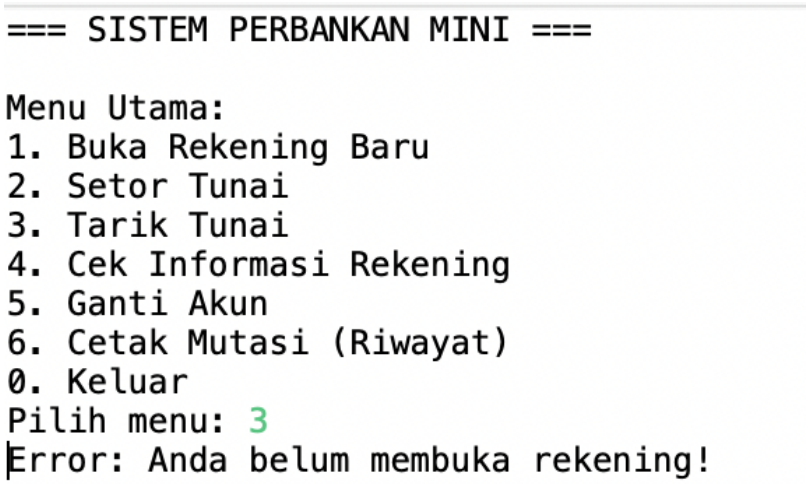
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Masukkan PIN: 235472
Akses Ditolak: PIN yang Anda masukkan salah!
```

Gambar 4.12 Output cetak mutasi dengan PIN salah (235472)

Hasil kedua pengujian menunjukkan bahwa riwayat transaksi hanya dapat ditampilkan setelah PIN yang benar diverifikasi.

c. Pengujian Tanpa Rekening Aktif

Sebagai pengujian tambahan, program dijalankan ulang dan menu 3 dipilih sebelum ada rekening yang dibuka sama sekali. Pengujian ini bertujuan membuktikan bahwa pengecekan akunAktif == null yang dipindahkan ke awal, sebelum PIN diminta, berhasil mencegah program mengalami crash.



Gambar 4.13 Output menu Tarik Tunai tanpa rekening aktif

Program menampilkan pesan "Error: Anda belum membuka rekening!" dengan baik tanpa memunculkan NullPointerException. Dengan demikian, penerapan enkapsulasi pada praktikum ini tidak hanya mengunci data internal class Transaksi dan Rekening, tetapi juga menjaga agar akses ke fitur tarik tunai dan cetak mutasi tetap aman dan tidak menimbulkan error.

BAB V – PRAKTIKUM 4: EKSPANSI PRODUK KEUANGAN (INHERITANCE)

Praktikum keempat membahas inheritance atau pewarisan, yaitu mekanisme yang memungkinkan sebuah class mewarisi atribut dan method dari class lain. Sampai praktikum sebelumnya, sistem hanya mengenal satu jenis rekening. Pada pertemuan ini sistem dikembangkan menjadi beberapa produk keuangan, yaitu Tabungan dan Giro, yang sama-sama berasal dari class Rekening sehingga kode dasar tidak perlu ditulis ulang. Seluruh kode pada pertemuan ini disimpan pada package pekan4.

5.1 Langkah Kerja Praktikum

a. Menyiapkan Class Rekening sebagai Superclass

Langkah pertama adalah menyiapkan class Rekening sebagai superclass atau kelas induk. Atribut nomorRekening, namaPemilik, dan pin tetap dibiarkan private karena bersifat sensitif dan tidak perlu diakses oleh kelas turunan. Sebaliknya, atribut saldo dan riwayatTransaksi diubah menjadi protected agar dapat diakses langsung oleh subclass, yaitu RekeningTabungan dan RekeningGiro, tanpa harus dibuka ke akses publik. Atribut protected hanya dapat diakses oleh class dalam package yang sama serta oleh subclass, sehingga enkapsulasi tetap terjaga sekaligus memberi ruang bagi pewarisan.

```
1  package pekan4;
2  import java.util.*;
3
4  public class Rekening {
5    // Bagian atas dari class Rekening.java
6      private String nomorRekening;
7      private String namaPemilik;
8      private String pin;
9
10     // Gunakan protected agar subscalss bisa mengaksesnya langsung
11     protected double saldo;
12     protected ArrayList<Transaksi> riwayatTransaksi;
13
14
15         //... (constructor dan method lainnya tetap sama seperti Modul 3)
16
```

Gambar 5.1 Kode class Rekening dengan atribut saldo dan riwayatTransaksi protected

b. Membuat Subclass RekeningTabungan

Class RekeningTabungan dibuat sebagai subclass dari Rekening menggunakan kata kunci extends, sehingga seluruh atribut dan method milik induknya otomatis diwarisi. Hubungan ini disebut relasi Is-A, yaitu sebuah Tabungan adalah sebuah Rekening. Class ini menambahkan satu atribut khusus, yaitu sukuBunga yang bersifat private.

Pada constructor, pemanggilan super(nomor, nama, saldoAwal, pinAwal) mengeksekusi constructor superclass untuk menginisialisasi data dasar rekening dan wajib berada di baris pertama. Baru setelah itu atribut sukuBunga diisi. Method tambahBungaAkhirBulan() menghitung bunga dari saldo saat ini dikalikan suku bunga dalam persen, menambahkannya ke saldo, lalu mencatat transaksi berjenis "Bunga" dengan ID berawalan "TRX-B-" ke dalam riwayatTransaksi. Method ini dapat mengakses saldo dan riwayatTransaksi secara langsung karena keduanya dideklarasikan protected di superclass, sehingga getter maupun setter tidak diperlukan.

```java
package pekan4;

public class RekeningTabungan extends Rekening {

    //Atribut Spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    //Constructor Subsclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;

    }

    public void tambahBungaAkhirBulan() {
         // Menghitung bunga
        // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat Riwayat Transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);

    }
}
```

Gambar 5.2 Kode class RekeningTabungan

Pengujian dilakukan dengan membuat rekening Tabungan melalui menu 1, memilih produk 1 (Tabungan Umum) dan suku bunga 5%. Program berhasil membuat objek RekeningTabungan atas nama dennis dengan nomor rekening 39284 dan saldo awal Rp100.000.000. Pesan "Rekening atas nama dennis berhasil dibuat" berasal dari constructor superclass Rekening yang dipanggil melalui super(), sehingga terbukti bahwa inisialisasi induk ikut berjalan saat subclass dibuat.

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 39284
Masukkan Nama Pemilik: dennis
Masukkan Saldo Awal: 100000000
Masukkan Pin (6 digit): 212223
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 5
Rekening atas nama dennis berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 1
```

Gambar 5.3 Output pembuatan rekening Tabungan Umum dengan suku bunga 5%

c. Membuat Subclass RekeningGiro

Class RekeningGiro juga merupakan subclass dari Rekening yang dibuat dengan extends. Fitur tambahannya adalah batasOverdraft, yaitu limit pinjaman yang bersifat private dan dilengkapi getter getBatasOverdraft() agar nilainya dapat dibaca dari luar class. Constructor-nya memanggil super(...) di baris pertama untuk menginisialisasi data dasar rekening, kemudian mengisi atribut batasOverdraft. Logika penarikan hingga batas overdraft belum diimplementasikan pada modul ini dan baru diselesaikan pada modul berikutnya.

```java
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft)
    {       // Memanggil inisialisasi dasar dari SuperClass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusu Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
    // (Catatan: Penarikan hingga limit Overdraft akan kita selesaikan di Modul 5)
}
```

Gambar 5.4 Kode class RekeningGiro

Pada pengujian, rekening Giro dibuat melalui menu 1 dengan memilih produk 2 (Giro Bisnis) dan batas overdraft Rp500.000. Program berhasil membuat objek RekeningGiro atas nama dennis_3020 dengan nomor 238473 dan menambahkannya ke daftar rekening sehingga total rekening menjadi 2. Hal ini menunjukkan bahwa dua jenis produk yang berbeda dapat disimpan dalam satu koleksi yang sama.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 238473
Masukkan Nama Pemilik: dennis_3020
Masukkan Saldo Awal: 15000000
Masukkan Pin (6 digit): 199193
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 500000
Rekening atas nama dennis_3020 berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 2
```

Gambar 5.5 Output pembuatan rekening Giro Bisnis dengan batas overdraft Rp500.000

5.2 Tugas/Latihan Praktikum

a. Tugas 1: Pembuatan Multi-Produk

Tugas pertama adalah menyesuaikan case 1 pada menu utama agar pengguna dapat memilih produk. Setelah nomor rekening, nama, saldo awal, dan PIN diisi, program menampilkan pilihan produk. Variabel baru dideklarasikan bertipe Rekening (superclass). Jika pengguna memilih produk 1, program meminta suku bunga lalu membuat objek RekeningTabungan. Jika memilih produk 2, program meminta batas overdraft lalu membuat objek RekeningGiro. Jika memilih selain itu, program menampilkan pesan "Pilihan produk tidak valid." dan keluar dari case.

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan Pin (6 digit): ");
    String pin = input.nextLine();

    System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
    System.out.print("Pilihan: ");
    int produk = Integer.parseInt(input.nextLine());

    Rekening baru; // tipe Superclass

    if (produk == 1) {
        System.out.print("Masukkan Suku Bunga (%): ");
        double bunga = Double.parseDouble(input.nextLine());
        baru = new RekeningTabungan(no, nama, saldo, pin, bunga); // Upcasting
    } else if (produk == 2) {
        System.out.print("Masukkan Batas Overdraft: ");
        double limit = Double.parseDouble(input.nextLine());
        baru = new RekeningGiro(no, nama, saldo, pin, limit); // Upcasting
    } else {
        System.out.println("Pilihan produk tidak valid.");
        break;
    }

    akunAktif = baru;
    rekeningAktif.add(baru);
    System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
    break;
```

Gambar 5.6 Kode case 1 (Buka Rekening Baru) dengan pilihan produk

Objek subclass yang disimpan ke dalam variabel bertipe Rekening merupakan upcasting yang terjadi secara implisit dan aman. Berkat upcasting ini,

objek Tabungan maupun Giro dapat disimpan bersama dalam satu ArrayList<Rekening>. Hasil pengujiannya telah ditampilkan pada Gambar 5.3 dan Gambar 5.5: kedua rekening berhasil dibuat dan total rekening bertambah dari 1 menjadi 2, yang membuktikan bahwa dua subclass berbeda dapat ditampung dalam satu koleksi bertipe superclass.

b. Tugas 2: Simulasi Waktu Akhir Bulan

Tugas kedua menambahkan menu baru, yaitu "7. Simulasi Akhir Bulan (Khusus Tabungan)". Pada case 7, program memeriksa tiga kondisi secara berurutan. Pertama, apakah akunAktif masih bernilai null. Kedua, apakah akunAktif merupakan objek RekeningTabungan, yang diperiksa menggunakan operator instanceof. Jika ya, dilakukan downcasting eksplisit ke tipe RekeningTabungan agar method tambahBungaAkhirBulan() dapat dipanggil, karena method khusus tersebut tidak dikenal oleh tipe Rekening. Ketiga, jika akun bukan Tabungan, program menolak dengan pesan kegagalan. Pengecekan instanceof sebelum downcasting penting untuk mencegah ClassCastException ketika objek aslinya ternyata bukan Tabungan.

```
case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan tab = (RekeningTabungan) akunAktif; // Downcasting
        tab.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
    }
    break;
```

Gambar 5.7 Kode case 7 (Simulasi Akhir Bulan) dengan instanceof dan downcasting

Percobaan pertama dilakukan dengan memilih menu 7 saat akun aktif adalah rekening Giro, yaitu akun yang dibuat paling akhir. Program menolak dengan pesan kegagalan karena hasil instanceof bernilai false untuk objek RekeningGiro, sehingga downcasting tidak dilakukan dan tidak terjadi error.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

Gambar 5.8 Output menu 7 saat akun aktif adalah rekening Giro

Selanjutnya akun aktif diganti ke rekening Tabungan nomor 39284 melalui menu 5.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 5
Masukkan nomor rekening yang ingin diaktifkan: 39284
Berhasil! Akun aktif sekarang atas nama dennis (No. Rekening: 39284)
```

Gambar 5.9 Output pergantian akun aktif ke rekening Tabungan (39284)

Percobaan kedua memilih menu 7 pada rekening Tabungan dengan saldo Rp100.000.000 dan suku bunga 5%. Bunga berhasil ditambahkan sebesar Rp5.000.000, sesuai perhitungan 5% dari saldo. Hal ini membuktikan bahwa downcasting berhasil dan method khusus milik RekeningTabungan dapat dijalankan.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 5.0% berhasil ditambahkan: Rp5000000.0
```

Gambar 5.10 Output simulasi akhir bulan pada rekening Tabungan

Riwayat transaksi kemudian diperiksa melalui menu 6 dengan PIN yang benar. Tercatat satu transaksi berjenis Bunga dengan ID berawalan TRX-B-, yang menunjukkan bahwa subclass berhasil menulis ke riwayatTransaksi milik superclass Rekening.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 6
Masukkan PIN: 212223
ID: TRX-B-1790594447084 | Jenis: Bunga | Nominal: Rp5000000.0
```

Gambar 5.11 Output cetak mutasi yang memuat transaksi Bunga

Terakhir, saldo diperiksa melalui menu 4. Saldo akhir tampil sebagai Rp1.05E8, yaitu cara Java menampilkan bilangan bertipe double yang bernilai besar dalam notasi ilmiah. Nilai 1.05E8 sama dengan 1,05 × 10^8 atau Rp105.000.000, sesuai dengan saldo awal Rp100.000.000 ditambah bunga Rp5.000.000. Tampilan ini sebenarnya kurang ramah bagi pengguna dan dapat diperbaiki dengan memformat keluaran menggunakan printf, tetapi perbaikan tersebut belum dikerjakan pada praktikum ini.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 39284
Nama Pemilik : dennis
Saldo Akhir  : Rp1.05E8
----------------------
```

Gambar 5.12 Output informasi rekening dengan saldo akhir Rp1.05E8

Secara keseluruhan, praktikum ini menunjukkan bahwa inheritance memungkinkan produk baru dibuat dengan menambahkan fitur khusus saja, sementara data dan perilaku dasar diwarisi dari Rekening. Upcasting dan downcasting melengkapinya dengan cara menyimpan berbagai produk dalam satu koleksi dan memanggil fitur khusus secara aman. Kode lengkap praktikum ini dapat dilihat pada repositori GitHub berikut.

https://github.com/shauqiakbar/PrakPBO_2511533020/tree/pekan4

BAB VI – PRAKTIKUM 5: TRANSAKSI DINAMIS (POLYMORPHISM)

Praktikum kelima membahas polymorphism, yaitu kemampuan satu perintah yang sama untuk menghasilkan perilaku berbeda tergantung jenis objek yang menjalankannya. Pada praktikum sebelumnya, Tabungan dan Giro sudah diturunkan dari class Rekening, tetapi keduanya masih memakai aturan penarikan yang sama. Pada pertemuan ini aturan penarikan Giro dibuat berbeda melalui overriding, lalu dimanfaatkan dalam fitur transfer antar rekening. Seluruh kode pada pertemuan ini disimpan pada package pekan5.

6.1 Langkah Kerja Praktikum

a. Refaktor Method tarikTunai() pada Class Rekening

Langkah pertama adalah mengubah tipe kembalian method tarikTunai() pada class Rekening dari void menjadi boolean. Method kini mengembalikan true apabila penarikan berhasil dan false apabila gagal, sehingga method lain, khususnya fitur transfer, dapat mengetahui hasil penarikan dan menentukan langkah selanjutnya. Terdapat tiga kemungkinan alur. Jika nominal kurang dari atau sama dengan 0, penarikan ditolak dan method mengembalikan false. Jika nominal melebihi saldo, penarikan juga ditolak dengan pesan saldo tidak mencukupi. Jika kedua syarat terpenuhi, saldo dikurangi, transaksi berjenis “Debit” dengan ID berawalan “TRX-T-” dicatat ke riwayatTransaksi, pesan sukses ditampilkan, lalu method mengembalikan true. Perlu dicatat bahwa pada refaktor ini pemeriksaan batas minimal penarikan Rp10.000 dari praktikum pertama digantikan oleh pemeriksaan nominal yang harus lebih dari 0.

```
// Mengubah void menjadi boolean
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
        return false;
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
        return true; // Transaksi sukses
    }
}
```

Gambar 6.1 Kode method tarikTunai() pada class Rekening dengan tipe kembalian boolean

Manfaat nilai kembalian ini terlihat saat fitur transfer diuji, yang dibahas lebih rinci pada subbab 6.2. Pada pengujian pertama, penarikan dari rekening Tabungan tab1 (nomor 2001) berhasil sehingga transfer dilanjutkan dan dana disetor ke rekening tujuan. Pada Gambar 6.2, saldo giro1 menjadi Rp700.000 karena sebelumnya sudah bernilai Rp-300.000 hasil pengujian pada langkah c.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1001
Masukkan nominal transfer: 1000000
Masukkan PIN: 222222
Tarik tunai Rp1000000.0 berhasil. Sisa saldo: Rp4000000.0
Setor tunai Rp1000000.0 berhasil. Saldo saat ini: Rp700000.0
Transfer Rp1000000.0 ke giro1 berhasil.
```

Gambar 6.2 Output transfer Rp1.000.000 dari Tabungan ke Giro yang berhasil

Pada pengujian kedua, nominal transfer Rp9.000.000 melebihi saldo tab1 yang tersisa Rp4.000.000. Method tarikTunai() mengembalikan false sehingga transfer ikut dibatalkan dengan pesan “Transfer Gagal”.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1001
Masukkan nominal transfer: 9000000
Masukkan PIN: 222222
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp4000000.0
Transfer Gagal: penarikan dari rekening asal tidak berhasil.
```

Gambar 6.3 Output transfer Rp9.000.000 yang gagal karena saldo tidak mencukupi

Kedua hasil tersebut menunjukkan bahwa nilai kembalian boolean berhasil dimanfaatkan oleh method transfer.

b. Overriding tarikTunai() pada Class RekeningGiro

Class RekeningGiro menimpa (override) method tarikTunai() milik superclass Rekening. Method yang di-override harus memiliki nama, parameter, dan tipe kembalian yang sama persis dengan milik induknya, serta diberi anotasi

@Override agar compiler memastikan method ini benar-benar menimpa method induk. Dengan anotasi tersebut, kesalahan penulisan nama method akan terdeteksi sebagai error.

Perbedaan logikanya terletak pada batas penarikan. Pada Rekening biasa, penarikan hanya diizinkan sampai sebesar saldo, sedangkan pada Giro penarikan diizinkan sampai sebesar saldo ditambah batasOverdraft, sehingga saldo boleh bernilai negatif selama masih berada dalam batas overdraft. Versi Giro juga mencatat transaksi dengan ID berawalan "TRX-G-" dan jenis "Debit Giro" agar dapat dibedakan dari penarikan rekening biasa.

```
package pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // IMPLEMENTASI POLIMORFISME (OVERRIDING)
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
            return false;
        }
        // Logika khusus Giro: boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        }
        else {
            saldo -= nominal; // Saldo bisa bernilai negatif di sini (memakan overdraft)

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

            System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }

    public double getBatasOverdraft() { return batasOverdraft; }
}
```

Gambar 6.4 Kode class RekeningGiro dengan method tarikTunai() yang di-override

Untuk pengujian, rekening Giro dibuat melalui menu 1 dengan nomor 1001 atas nama giro1, saldo awal Rp100.000, PIN 111111, produk 2 (Giro Bisnis), dan batas overdraft Rp500.000.

```
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1001
Masukkan Nama Pemilik: giro1
Masukkan Saldo Awal: 100000
Masukkan Pin (6 digit): 111111
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 500000
Rekening atas nama giro1 berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 1
```

Gambar 6.5 Output pembuatan rekening Giro giro1 dengan batas overdraft Rp500.000

c. Uji Coba Pengikatan Dinamis

Pada langkah ini program dijalankan tanpa mengubah satu baris pun kode pada menu 3 (Tarik Tunai). Menu 3 tetap memanggil akunAktif.tarikTunai(tarik), sedangkan versi method yang dijalankan ditentukan oleh jenis objek yang tersimpan di dalam akunAktif ketika program berjalan.

Percobaan pertama menarik Rp400.000 dari rekening Giro yang hanya bersaldo Rp100.000. Penarikan berhasil dan saldo menjadi Rp-300.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 3
Masukkan PIN: 111111
Masukkan nominal tarik: 400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0
```

Gambar 6.6 Output tarik tunai Rp400.000 dari rekening Giro bersaldo Rp100.000

Seandainya objek ini berupa Rekening biasa, penarikan tersebut pasti ditolak karena melebihi saldo. Namun karena objek aslinya adalah RekeningGiro, Java secara otomatis memanggil versi tarikTunai() milik RekeningGiro. Inilah yang disebut dynamic method dispatch, yaitu JVM memilih method yang dijalankan

berdasarkan tipe objek sebenarnya pada saat runtime, bukan berdasarkan tipe variabel referensinya.

Percobaan kedua menarik Rp300.000 lagi dari rekening yang sama. Penarikan ditolak karena dari batas overdraft Rp500.000 sudah terpakai Rp300.000, sehingga sisa yang boleh ditarik hanya Rp200.000, sesuai pesan "Maksimal penarikan: Rp200000.0". Hal ini membuktikan bahwa batas overdraft ditegakkan oleh method tarikTunai() versi Giro.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 3
Masukkan PIN: 111111
Masukkan nominal tarik: 300000
Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp200000.0
```

Gambar 6.7 Output tarik tunai Rp300.000 yang ditolak karena melebihi sisa overdraft

6.2 Tugas/Latihan Praktikum

a. Method transfer() pada Class Rekening

Tugas pada praktikum ini adalah membuat fitur transfer antar rekening. Method transfer() menerima parameter bertipe Rekening untuk rekening tujuan. Karena RekeningTabungan dan RekeningGiro adalah subclass dari Rekening, parameter ini dapat menerima objek dari kedua jenis rekening, sehingga method tidak perlu mengetahui tipe persis rekening tujuan. Inilah yang disebut parameter polimorfik.

Logika method berjalan dalam tiga tahap sesuai modul. Pertama, PIN diperiksa menggunakan otentikasi(pinInput) dan transfer dibatalkan jika PIN salah. Kedua, this.tarikTunai(nominal) dipanggil pada rekening asal. Jika rekening asal adalah Giro, versi override yang dipakai sehingga aturan overdraft berlaku secara otomatis. Ketiga, nilai boolean hasil tarikTunai() diperiksa; jika true, tujuan.setorTunai(nominal) dipanggil agar dana masuk ke rekening tujuan. Selain yang diminta modul, ditambahkan pengecekan agar rekening tidak dapat mentransfer ke dirinya sendiri.

```java
// Parameter polimorfik: tujuan bisa bertipe Tabungan atau Giro
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        return;
    }
    if (tujuan == this) {
        System.out.println("Transfer Gagal: Rekening tujuan tidak boleh sama dengan rekening asal.");
        return;
    }
    if (this.tarikTunai(nominal)) { // overriding: kalau this = Giro, aturan overdraft dipakai
        tujuan.setorTunai(nominal);
        System.out.println("Transfer Rp" + nominal + " ke " + tujuan.getNamaPemilik() + " berhasil.");
    } else {
        System.out.println("Transfer Gagal: penarikan dari rekening asal tidak berhasil.");
    }
}
```

Gambar 6.8 Kode method transfer() pada class Rekening

b. Integrasi Menu 8 pada main.java

Menu "8. Transfer ke Rekening Lain" ditambahkan pada daftar menu utama. Pada case 8, program memeriksa apakah akunAktif masih null. Jika sudah ada rekening aktif, pengguna diminta memasukkan nomor rekening tujuan, lalu program mencari nomor tersebut di dalam daftar rekening menggunakan perulangan for-each dan method cocokDenganNomor(). Jika tidak ditemukan, program menampilkan pesan kegagalan. Jika ditemukan, pengguna diminta memasukkan nominal transfer dan PIN, kemudian program memanggil akunAktif.transfer(akunTujuan, nominalTf, pinTf). Perintah input.nextLine() setelah nextDouble() diperlukan untuk membersihkan sisa baris baru di buffer sebelum PIN dibaca.

```java
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan Nomor Rekening Tujuan: ");
        String noTujuan = input.nextLine();
        Rekening akunTujuan = null;
        for (Rekening r : rekeningAktif) {
            if (r.cocokDenganNomor(noTujuan)) {
                akunTujuan = r;
                break;
            }
        }
        if (akunTujuan == null) {
            System.out.println("Gagal: Nomor rekening tujuan " + noTujuan + " tidak ditemukan.");
        } else {
            System.out.print("Masukkan nominal transfer: ");
            double nominalTf = input.nextDouble();
            input.nextLine();
            System.out.print("Masukkan PIN: ");
            String pinTf = input.nextLine();
            akunAktif.transfer(akunTujuan, nominalTf, pinTf);
        }
    }
    break;
```

Gambar 6.9 Kode case 8 (Transfer ke Rekening Lain) pada main.java

c. Pengujian Fitur Transfer

Sebagai persiapan, dibuat rekening Tabungan nomor 2001 atas nama tab1 dengan saldo awal Rp5.000.000, PIN 222222, dan suku bunga 5%. Total rekening menjadi 2 dan rekening ini menjadi akun aktif.

```
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 2001
Masukkan Nama Pemilik: tab1
Masukkan Saldo Awal: 5000000
Masukkan Pin (6 digit): 222222
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 5
Rekening atas nama tab1 berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 2
```

Gambar 6.10 Output pembuatan rekening Tabungan tab1

Percobaan pertama, yaitu transfer Rp1.000.000 dari tab1 ke giro1 dengan PIN yang benar, telah ditampilkan pada Gambar 6.2. Dana ditarik dari rekening asal sehingga sisa saldo menjadi Rp4.000.000, kemudian disetor ke rekening tujuan sehingga saldo giro1 menjadi Rp700.000, yaitu Rp-300.000 ditambah Rp1.000.000. Hasil ini membuktikan bahwa method transfer() dapat menerima rekening tujuan bertipe Giro melalui parameter bertipe Rekening.

Pada percobaan kedua, transfer dilakukan dengan PIN yang salah (000000). Transfer langsung dibatalkan pada tahap otentikasi sehingga tidak ada dana yang ditarik maupun disetor.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 1001
Masukkan nominal transfer: 1000000
Masukkan PIN: 000000
Akses Ditolak: PIN yang Anda masukkan salah!
```

Gambar 6.11 Output transfer dengan PIN salah (000000)

Percobaan ketiga, yaitu transfer Rp9.000.000 yang melebihi saldo rekening asal, telah ditampilkan pada Gambar 6.3. Penarikan pada rekening asal gagal dan mengembalikan false sehingga setor ke rekening tujuan tidak dilakukan. Dengan demikian dana tidak hilang dari rekening asal maupun bertambah di rekening tujuan.

Percobaan keempat mencoba transfer ke nomor rekening yang tidak terdaftar, yaitu 9999. Program menampilkan pesan bahwa nomor rekening tujuan tidak ditemukan dan tidak meminta nominal maupun PIN.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 9999
Gagal: Nomor rekening tujuan 9999 tidak ditemukan.
```

Gambar 6.12 Output transfer ke nomor rekening tujuan yang tidak terdaftar (9999)

Percobaan kelima membalik arah transfer, yaitu Rp1.100.000 dari rekening Giro (giro1, saldo Rp700.000) ke rekening Tabungan (tab1) dengan PIN 111111. Transfer berhasil meskipun nominalnya melebihi saldo Giro, karena method this.tarikTunai() yang dipanggil di dalam transfer() adalah versi override milik RekeningGiro. Saldo Giro menjadi Rp-400.000, masih dalam batas overdraft Rp500.000, sedangkan saldo Tabungan bertambah menjadi Rp5.100.000.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan Nomor Rekening Tujuan: 2001
Masukkan nominal transfer: 1100000
Masukkan PIN: 111111
Tarik tunai Giro Rp1100000.0 berhasil. Saldo saat ini: Rp-400000.0
Setor tunai Rp1100000.0 berhasil. Saldo saat ini: Rp5100000.0
Transfer Rp1100000.0 ke tab1 berhasil.
```

Gambar 6.13 Output transfer Rp1.100.000 dari rekening Giro ke Tabungan

Pengujian ini memperlihatkan bahwa polimorfisme bekerja di dalam method transfer(). Kode transfer yang sama menghasilkan aturan penarikan yang berbeda tergantung jenis rekening asal: transfer Rp1.100.000 dari Tabungan bersaldo Rp700.000 pasti gagal, sedangkan dari Giro berhasil.

Terakhir, riwayat transaksi rekening Giro diperiksa melalui menu 6 dengan PIN yang benar.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 6
Masukkan PIN: 111111
ID: TRX-G-1792932377420 | Jenis: Debit Giro | Nominal: Rp400000.0
ID: TRX-S-1792932452537 | Jenis: Kredit | Nominal: Rp1000000.0
ID: TRX-G-1792932581338 | Jenis: Debit Giro | Nominal: Rp1100000.0
```

Gambar 6.14 Output cetak mutasi rekening Giro setelah rangkaian pengujian

Riwayat mencatat tiga transaksi. Transaksi pertama adalah penarikan Rp400.000 pada langkah c, transaksi kedua adalah setoran Rp1.000.000 dari transfer Tabungan ke Giro, dan transaksi ketiga adalah penarikan Rp1.100.000 untuk transfer ke Tabungan. Kedua penarikan tercatat sebagai jenis Debit Giro dengan ID berawalan TRX-G-, yang membuktikan bahwa versi override milik RekeningGiro yang dijalankan, sedangkan setoran tercatat sebagai Kredit dengan ID berawalan TRX-S-.

Secara keseluruhan, praktikum ini menunjukkan bahwa overriding memungkinkan subclass mengubah perilaku yang diwarisi, sementara dynamic method dispatch memastikan versi yang tepat dipilih saat program berjalan tanpa perlu mengubah kode pemanggilnya. Kode lengkap praktikum ini dapat dilihat pada repositori GitHub berikut.

https://github.com/shauqiakbar/PrakPBO_2511533020/tree/pekan5

BAB VII – KESIMPULAN

Bab ini merangkum perkembangan sistem perbankan mini dari praktikum pertama hingga kelima, menyimpulkan konsep yang telah dipelajari, serta mencatat keterbatasan yang masih ada pada program.

7.1 Perkembangan Sistem

Selama lima pertemuan, sistem berkembang dari satu class sederhana menjadi aplikasi perbankan konsol yang mengenal beberapa produk rekening, riwayat transaksi, otentikasi PIN, dan transfer antar rekening. Setiap pertemuan menambahkan satu konsep PBO tanpa membuang hasil pertemuan sebelumnya, sebagaimana diringkas pada Tabel 7.1.

No	Pekan	Penambahan pada Sistem	Hasil Utama
1	pekan1	Class Rekening, constructor, setor dan tarik tunai, ArrayList untuk multi-akun	Sistem dapat membuat dan mengelola lebih dari satu rekening
2	pekan2	Class Transaksi dan relasi one-to-many melalui riwayatTransaksi	Setiap setoran dan penarikan tercatat sebagai riwayat
3	pekan3	Atribut private, getter tanpa setter, PIN, dan method otentikasi()	Saldo dan riwayat terlindungi; tarik tunai dan mutasi memerlukan PIN
4	pekan4	Rekening sebagai superclass, RekeningTabungan dan RekeningGiro, protected, upcasting, downcasting	Dua produk tersimpan dalam satu koleksi; bunga akhir bulan khusus Tabungan
5	pekan5	tarikTunai() bertipe boolean, override pada RekeningGiro, method transfer()	Aturan overdraft berlaku otomatis; transfer berjalan lintas jenis rekening

Tabel 7.1 Perkembangan sistem dari pekan 1 sampai pekan 5

7.2 Kesimpulan

Berdasarkan seluruh praktikum, dapat disimpulkan bahwa pemrograman berorientasi objek memungkinkan sebuah sistem dikembangkan secara bertahap dan terstruktur. Class dan constructor pada praktikum pertama menjadi fondasi untuk membuat objek rekening beserta data awalnya. Relasi asosiasi pada

praktikum kedua menunjukkan bahwa satu objek dapat memiliki banyak objek lain, sehingga satu rekening mampu menyimpan banyak transaksi.

Enkapsulasi pada praktikum ketiga melindungi data penting dengan menyembunyikan atribut dan membuka akses hanya melalui method yang terkontrol, misalnya otentikasi() yang membandingkan PIN tanpa pernah menampilkannya. Inheritance pada praktikum keempat memungkinkan produk Tabungan dan Giro dibuat tanpa menulis ulang kode dasar rekening, sedangkan upcasting dan downcasting memungkinkan keduanya disimpan dalam satu koleksi dan tetap dapat memanggil fitur khusus secara aman. Polymorphism pada praktikum kelima melengkapi rangkaian ini: dengan overriding dan dynamic method dispatch, kode pemanggil yang sama menghasilkan perilaku yang sesuai dengan jenis objeknya, seperti terlihat pada penarikan Giro yang boleh melampaui saldo hingga batas overdraft.

7.3 Repositori Kode

Seluruh kode program dari kelima praktikum, yang dipisahkan ke dalam package pekan1 sampai pekan5, tersimpan pada repositori GitHub berikut.

https://github.com/shauqiakbar/PrakPBO_2511533020

7.4 Link Video Penjelasan Youtube

https://youtu.be/F6pkp3UFEAs